We Propose 1

- **Integrating Authenticated Secret Sharing (ASS)**
  - Each secret share is paired with a one-time MAC/tag so any modification is detected during later secure operations
- Prevents tampering of intermediate results
  - A party cannot flip its share of $[out_i]$ (Hamming pass/fail) or of payload $[z_i]$; forged tags fail verification → abort.
- Enforces cross-round consistency
  - Inconsistent or replayed shares across the k rounds are detected when MAC checks are applied in subsequent computations
- Raises security beyond semi-honest (with abort)
  - Moves protocol **towards malicious security**

CONFIDENTIAL

Where to Add ASS

- After $F_{ss}PSI$: return authenticated shares of intersection payloads $[z]$.
- After $F_{ss}HamCom$: return authenticated shares of $[out]$.
- Inside $F_{ss}VMult$: operate on authenticate $[out]$ and $[z]$; verify tags before/after multiplication; output authenticated $[\tilde{z}]$.
- Final opening: reconstruct only for survivors whose tags verified across the pipeline.

CONFIDENTIAL

We Propose 2

- **Integrating a lightweight Projection-Consistency Checks**
  - For each round j, prove the submitted projection a_{I_j} truly comes from the original bitstring a without revealing either. We use a **probabilistic linear check** that verifies all projected positions at once.
- Prevents projection-stage cheating
  - **No forged bits**: A party cannot inject arbitrary bits into a_{I_j}
  - **No payload swap/position swap**: Wrong (a_{I_j})pair or permuted indices breaks the equality check
  - **No targeted collision manipulation**: A cheating party cannot fake a projection match or hide a real projection match

CONFIDENTIAL

Where to add Projection-Consistency Checks

- immediately after **projection** and before feeding any projected values into $\boldsymbol{F_{ssPSI}}$

1. Projection step (Algorithm 1, Step 4)
   - This is exactly where malicious cheating can happen
   - A malicious party could submit fake projection bits that don't match their original bitstring
2. **Insert Projection-Consistency Check (new step)**
   - **Verify that the submitted projection truly corresponds to the original string**
   - **If the check fails → abort the protocol (prevents projection-level cheating)**
3. After passing the check → continue to F_{ssPSI} (Algorithm 1, Step 5)
   - Only valid, correctly-formed projections feed into secret-shared PSI

CONFIDENTIAL

How Projection-Consistency Works

- For projection mask $I_j = \{i_1, \dots, i_m\}$, parties jointly generate random coefficients: $r_1, r_2, \dots, r_m$
- Compute two secret-shared "checksums"
  - Checksum of claimed projection:
    $$L_1 = \sum r_t \cdot a_{I_j}[t]$$
  - Checksum of the true original bits at position I_j:
    $$L_2 = \sum r_t \cdot a[i_t]$$
- Securely compare $L_1 == L_2$ (can reuse $F_{ssHamCom}$)
- If equal → projection accepted, if not → cheating detected → abort

CONFIDENTIAL

Solution: Sorted 3-Gram Bit-Vector

- **Pre-processing**
  - Split into tokens and sort tokens alphabetically
  - **"Warren Wang" → ["WANG", "WARREN"] → "WANGWARREN"**
- **3-Gram Extraction**
  - Slide a window of 3 characters over the normalised string
  - **WANGWARREN → WAN, ANG, NGW, GWA, WAR, ARR, RRE, REN**
- **Hashing to Bit Positions**
  - Compute hash for each 3-gram
  - index = Hash(gram) mod n
  - Use a deterministic hash (e.g. SHA256)
- **Bit-Vector Construction**
  - Set bit at each computed index to 1
  - Result: deterministic binary feature vector

CONFIDENTIAL

Solving the FM-12 Rule with Sorted 3-Gram Bit-Vector

- **Can address** FM-12 cases, but not by default
- Sorted 3-Gram Bit-Vector **+ small extensions**
- Example of **Only** Sorted 3-Gram Bit-Vector
  - Simon Baron Cohen VS Simon Cohen
  - Normalisation + Sorting Token

    ["SIMON", "BARON", "COHEN"] → BARONCOHENSIMON

    ["SIMON", "COHEN"] → COHENSIMON
  - 3-grams
    - BAR ARO RON ONC NCO COH OHE HEN ENS NSI SIM IMO MON
    - COH OHE HEN ENS NSI SIM IMO MON
- The shorter name's grams are a subset of the longer name's grams. The difference is exactly the grams contributed by **BARON**

CONFIDENTIAL

Technique 1: Tail-Token Preservation

- When a compound token **X-Y** appears, explicitly preserve the tail token **Y** in the encoding
- Encoding Steps
  - Normalise
  - Apply Tail-Token Rule
    - If token = X-Y
    - Encode both XY and Y
  - Sort Tokens + Concatenate
    - Alphabetically sort tokens
    - Concatenate into a single string
  - 3-Gram Bit-Vector Construction
    - Extract 3-grams
    - Hash each 3-gram to a bit index
    - Combine via bitwise OR

CONFIDENTIAL

Technique 2: Token-Level OR Composition

- Encode each token independently, then combine by bitwise OR
- Token order and position become irrelevant
- Encoding Steps
  - Normalise and Tokenise
  - Per-Token 3-Gram Encoding
    - Encode each token into its own 3-gram bit-vector
  - OR Combine Token Vectors
    - Final vector is the OR of all token vectors

CONFIDENTIAL